

Algorithms and Data Structures

27.06.2022

Exercise 1 (1.0 points)

Given the following array of integer values, sort it in descending order using the merge sort procedure.

1 2 8 3 5 7 13 12 11 10

Display the content of the array just before the last (and conclusive) merge step (the one delivering the final and sorted array). Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response.

Exercise 2 (1.0 points)

Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order, thus from the initial array generate the right and the left partitions.

3 13 5 8 2 1 7 11 15 9

Report 3 integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. This is an example of response format: 13 1 10

Exercise 3 (1.0 points)

Consider a binary tree whose visits return the following sequences.

Pre-order:	A	B	E	F	G	H	L	C	D	I
In-order:	E	F	B	G	L	H	A	I	D	C
Post-order:	F	E	L	H	G	B	I	D	C	A

Report the sequence of keys stored on the leaves of the tree, moving on the tree from left to right. Please, report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. This is an example of the response format: A X C Y etc.

Exercise 4 (1.0 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=19$. Insertions occur character by character using open addressing with linear probing. Each character is identified by its index in the English alphabet (i.e., $A=1, \dots, Z=26$). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

L M N L N M

Indicate in which elements are placed the last three letters of the sequence, i.e., L, N, and M, in this order. Please, report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response format: 3 4 11

Exercise 5 (4.0 points)

A set of strings, each one including an undefined number of any ASCII characters, are used as keys to access a hash table of size M, implemented with linear chaining. Report the C implementation of a proper hash function receiving the string as a parameter and returning the index of the hash table.

Describe the logic of the algorithm and specify how it is possible to avoid overflows during the computation of the hash function. Could you suggest a different implementation for the case in which the strings include only small alphabetic letters? Motivate all choices.

Exercise 6 (1.0 points)

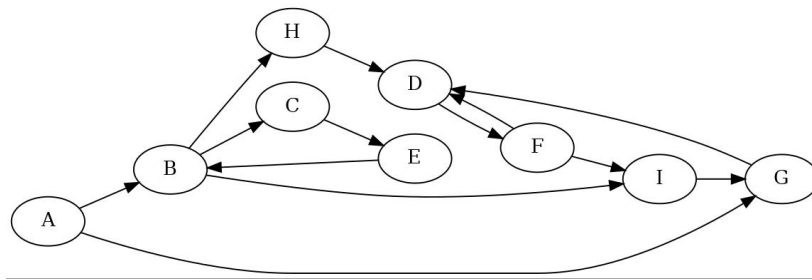
Given the following sequence of integers stored into an array, turn it into a heap, assuming to use an array as an underlying data structure. Assume that, in the end, the largest value is stored at the heap's root. Then, execute the first two steps of heapsort on the heap built at the previous step.

2 1 6 3 13 15 21 9

Report the final content of the entire array at the end of the above process. Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Exercise 7 (1.0 points)

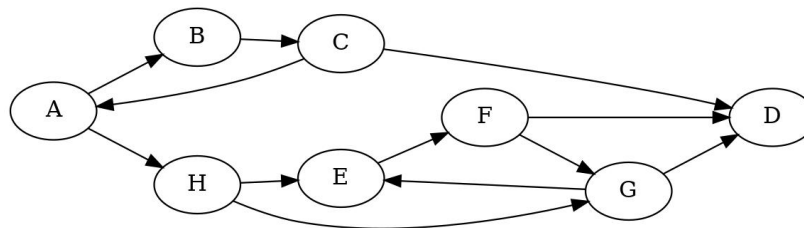
Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please, indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response: 15 13 2 16 8 etc.

Exercise 8 (1.0 points)

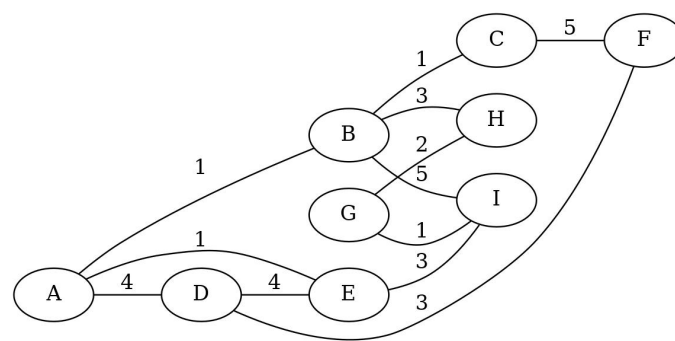
Given the following directed graph, represent the reverse graph and find all strongly connected components.



Compute all strongly connected components in the graph. Report them in alphabetic order and within each component indicates vertices in alphabetic order (i.e., ACZ and not CAZ or ZAC). Separate components with a single space. No other symbols must be included in the response. This is an example of the response format: ABC EF XY etc.

Exercise 9 (1.0 points)

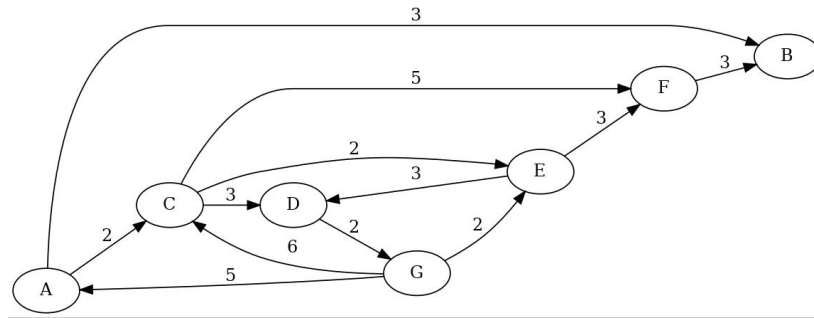
Given the following undirected and weighted graph find a minimum spanning tree using Kruskal algorithm.



Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. This is an example of the response format: 13

Exercise 10 (1.0 points)

Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node A with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please, indicate the shortest path to all vertices sorted in alphabetic order (i.e, display the shortest path for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Exercise 11 (2.0 points)

Analyze the following program and indicate the exact output generated.

```

#define N 64
void f (int);
int main(void) {
    f (22);
    return (1);
}
void f (int n) {
    int vet[N], i;
    i = 0;
    do {
        vet[i++] = n % 2;
        n = n / 2;
    } while (n != 0);
    while (i>0) {
        printf ("%d", vet[--i]);
    }
    return;
}

```

Report the exact program output with no extra symbols.

Exercise 12 (2.0 points)

The following function computes simple arrangements.

```

int arrangement_simple(int *value, int *solution, int *mark, int n, int pos, int k, int count) {
    int i;
    if (pos >= k) {
        fprintf(stdout, "{ ");
        for (i=0; i<k; i++) {
            fprintf(stdout, "%d ", solution[i]);
        }
        fprintf(stdout, "}");
        return count+1;
    }
    for (i=0; i<n; i++) {
        if (mark[i] == 0) {
            mark[i] = 1;
            solution[pos] = value[i];
            count = arrangement_simple(value, solution, mark, n, pos+1, k, count);
            mark[i] = 0;
        }
    }
    return count;
}

```

Indicates which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. To generate the arrangement with repetition the array mark is useless and can be erased from the implementation.
2. To generate the simple combination the array mark is required but the main recursion loop does not have to start from zero but from an initial value "start".
3. The simple arrangements of n distinct objects of class k are $n \cdot n \cdot n \cdot n \dots$ (k times).
4. To generate the arrangement with repetition the termination condition must be modified.
5. The simple arrangements of n distinct objects of class k are $n!/(n-k)!$.

6. To generate the simple combination the array mark is not required and the main recursion loop does not have to start from zero but from an initial value "start".

Exercise 13 (2.0 points)

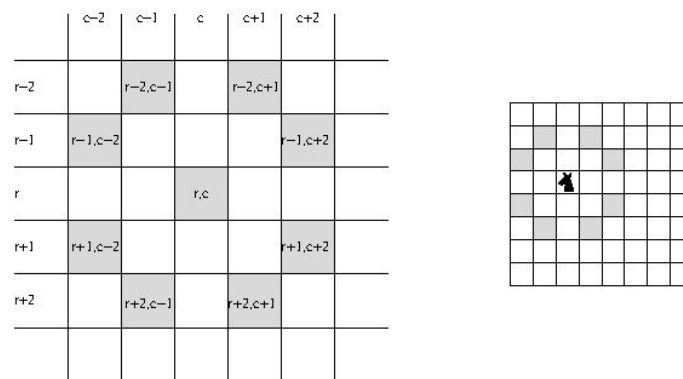
Analyze the following recursive program.

```
void f1 (int);
void f2 (int);
void f1 (int n) {
    if (n<=0) {
        return;
    }
    printf ("1");
    f2 (n-2);
    return;
}
void f2 (int n) {
    if (n<=0) {
        return;
    }
    printf ("2");
    f1 (n+1);
    return;
}
int main () {
    f2(5);
    return 1;
}
```

Report the exact program output with no other symbols.

Exercise 14 (3.0 points)

In the chess game, a knight moves on the board drawing L letters as in the left-hand side figure.



Moreover, in the chess game each piece (pawn, knight, bishop, rook, queen, and king) has a specific value (e.g., 1 for a pawn, 3 for a knight, etc.). Empty cells are assigned a zero value.

Write the iterative (non-recursive) C function:

```
void knight (int mat[N][N], int &r, int &c);
```

which, given the integer matrix mat, returns in r and c the coordinates of the empty cell of the matrix in which a knight can be placed such that the sum of the values of all pieces it can reach in one single move is maximum. N is a constant value (e.g., 8, 10, etc.).

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 15 (5.0 points)

Write the function

```
void mul (int *v1, int *v2, int n, int **pv);
```

which multiplies the decimal number stored in the array v1 by the decimal number stored in v2. Numbers are stored in the arrays one digit for each element. The value of n is the number of digits of the two numbers (and it can be

an extremely large value, allowing `v1` and `v2` to represent extremely large numbers). The function must apply the standard sum-and-shift algorithm, taking care of the carry values, as indicated by the following example:

```

      0 3 2 x
      2 4 3 =
-----
0 0 0 0 9 6 +
0 0 1 2 8   =
0 0 6 4
-----
0 0 7 7 7 6

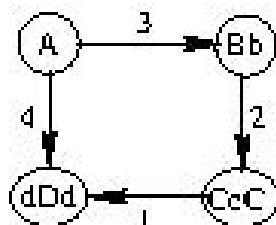
```

Notice that, to avoid overflow, the final product has to be represented on $2 \cdot n$ digits. Function `mult` has to allocate the array referenced by the pointer `pv`, and it has to return the result in this array.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 16 (9.0 points)

A weighted directed graph is stored as a list of list whose C types must be defined from the candidate. The following figure illustrates an example of such a graph.



Suppose that the graph is already stored into the main memory using the data structure previously defined. Write two C functions:

- ... `isRegular (...)` to verify whether the graph is a regular graph. An oriented graph is said to be regular if each vertex has the same number of neighbors, i.e., if and only if all vertices have the same input and output degrees and those values are the same for all vertices.
- ... `findSimplePath (...)` to read a vertex identifier and find the simple path (a path with no cycles) with a source in that vertex for which the sum of the weights is larger. For such a path, the program has to print out on standard output the list of weights, the lists of arcs, and the total weight.

Implement both functions by choosing the proper function arguments.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.